

SYSTEM ANALYSIS & DESIGN

AI-Powered Predictive Maintenance for Industrial Equipment

Turbofan Engine RUL Prediction • Acoustic Anomaly Detection

Using LSTM Deep Learning, TS2Vec Transfer Learning & Autoencoder-Based Detection

Team Members

Alaa Mohamed Hamed Awad

Nada Ahmed Jalal

Taghreed Mohammed Abdelkader

Rowan Nasser Abbas

Executive Summary

This document presents the System Analysis and Design specification for an AI-Powered Predictive Maintenance System comprising two integrated modules: a Turbofan Engine Remaining Useful Life (RUL) Prediction Module and an Acoustic Anomaly Detection Module.

The RUL Prediction Module is developed using the NASA CMAPSS FD004 dataset. It employs regime-aware preprocessing with K-Means clustering ($k=6$), followed by two deep learning pipelines: a Baseline LSTM and a TS2Vec transfer-learning-enhanced LSTM. Both are evaluated using RMSE, MSE, MAE, MAPE, and R^2 metrics.

The Acoustic Anomaly Detection Module uses the MIMII dataset (valves, pumps, fans, slide rails). Audio signals are preprocessed to extract MFCC and YAMNet embeddings, which are fed into a deep autoencoder trained exclusively on normal data. Anomalies are detected via reconstruction error thresholding, with majority voting applied at the file level.

Both modules are integrated into a unified deployment pipeline using FastAPI and Gradio, hosted on Hugging Face Spaces, with MLflow for experiment tracking.

1. Problem Statement & Objectives

1.1 Problem Statement

Unforeseen failures of industrial equipment — including turbofan engines and rotating machinery — present significant safety risks and result in costly unscheduled maintenance and operational downtime. Calendar-based maintenance schedules are inherently inefficient: applied prematurely, they incur unnecessary cost; applied too late, they risk in-service failure.

The system addresses two complementary dimensions of predictive maintenance:

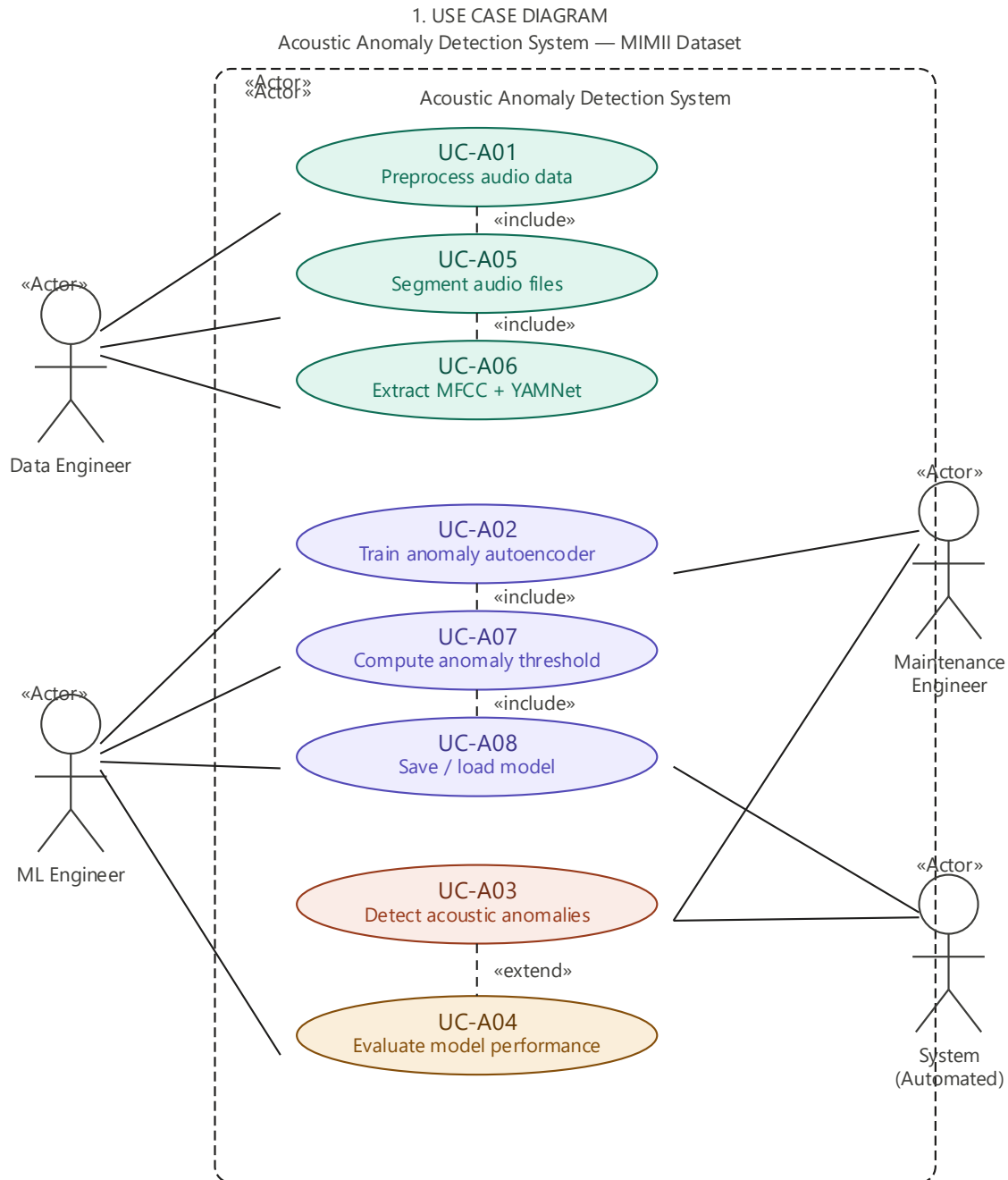
- **RUL Prediction:** The NASA CMAPSS FD004 sub-dataset was selected for its high complexity — engines operate across six distinct regimes under two simultaneous fault modes (HPC degradation and fan degradation). Without accounting for operating regime, predictive models conflate regime-induced sensor variation with true degradation signals.
- **Acoustic Anomaly Detection:** The MIMII dataset captures machine sound from four equipment types (valves, pumps, fans, slide rails). Different machine types exhibit distinct acoustic characteristics; overlapping normal/abnormal distributions make unsupervised detection challenging.

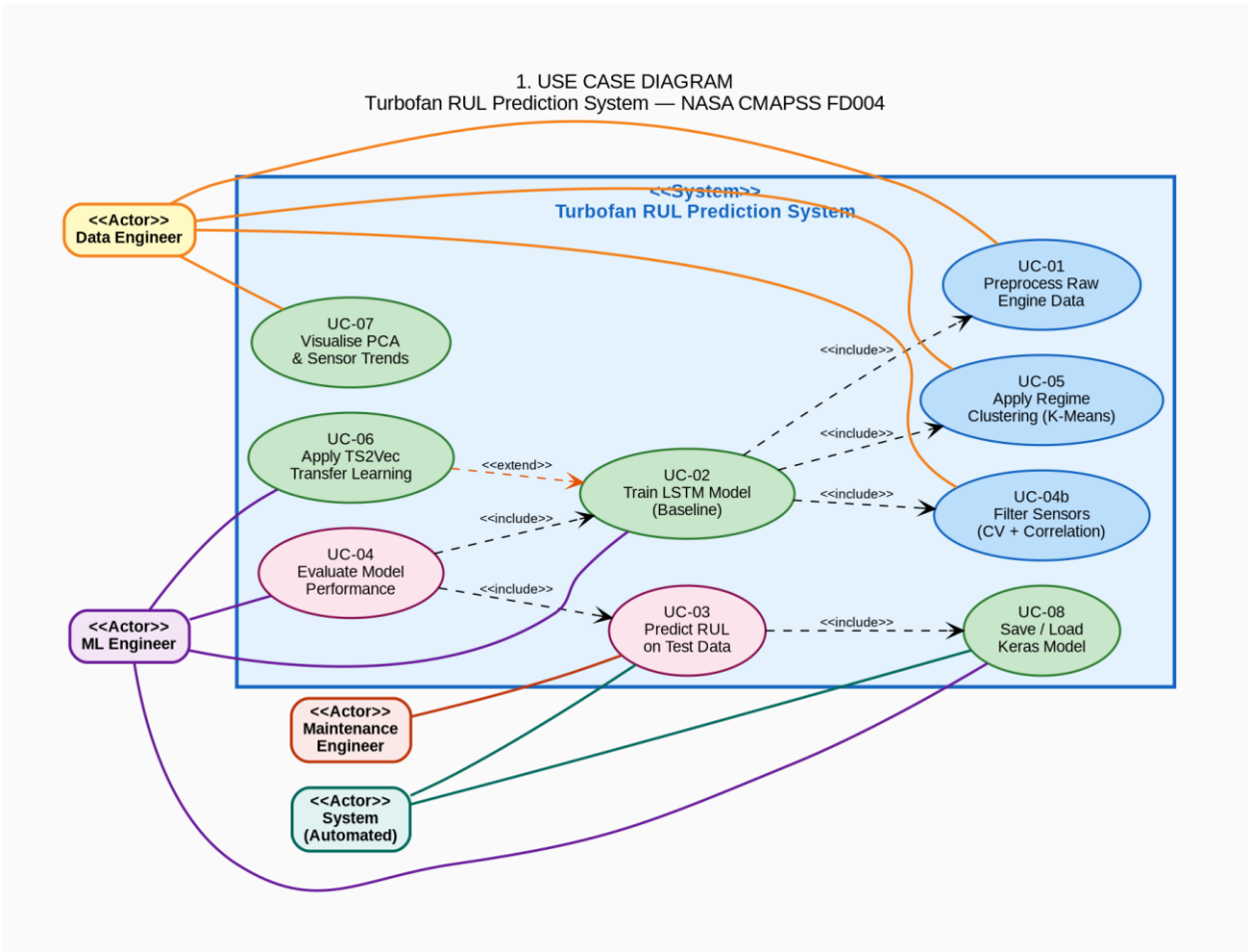
1.2 Project Objectives

- Implement an end-to-end RUL prediction pipeline based on the NASA CMAPSS FD004 dataset.
- Apply regime-aware preprocessing using K-Means clustering to address non-stationarity from multi-condition operation.
- Employ CV-based sensor filtering combined with regime-wise Pearson correlation analysis for feature selection.
- Train a stacked LSTM network using a unit-wise 80/20 train/validation split to prevent temporal data leakage.
- Implement TS2Vec self-supervised transfer learning as a feature extraction stage prior to LSTM training.
- Build an acoustic anomaly detection system using MFCC and YAMNet embeddings with a deep autoencoder.
- Apply reconstruction-error-based thresholding (statistical and percentile-based) with majority voting for anomaly classification.
- Evaluate all models using industry-standard regression and classification metrics.
- Deploy the integrated system via FastAPI, Gradio, and Hugging Face Spaces with MLflow experiment tracking.

2. Use Case Diagram & Descriptions

The use case model identifies the primary actors interacting with the AI-Powered Predictive Maintenance System and the key functional interactions across both modules.





2.1 System Actors

Actor	Role	System Interaction
Data Engineer	Prepares and validates raw sensor and audio data	Uploads CMAPSS/MIMII files; triggers and monitors preprocessing pipelines
ML Engineer	Designs, trains, and evaluates models	Configures hyperparameters, selects model architectures, interprets metrics, commits experiments to version control
Maintenance Engineer	Consumes RUL predictions and anomaly alerts for operational decisions	Queries the system with new engine sensor readings or audio files; receives RUL estimates or anomaly classification
System (Automated)	Executes model inference on new sensor windows and audio files	Preprocesses input using saved scalers/regime assignments; runs LSTM inference or autoencoder reconstruction; outputs predictions

2.2 Use Case Descriptions — RUL Module

UC-01: Preprocess Raw Engine Data

Actor: Data Engineer

Precondition: Raw CMAPSS .txt files (train_FD004, test_FD004, RUL_FD004) are available.

Main Flow:

- Read space-delimited text files (unit_number, time_cycles, op_setting_1–3, sensor_1–23).
- Drop sensor_22 and sensor_23 (null in FD004). Compute $RUL = \max_cycles_per_unit - time_cycles$.
- Apply MinMax scaling to op_settings [-1, 1] (fit on train, transform-only on test).
- Cluster op_settings into k=6 using K-Means (n_init=50, random_state=42).
- Remove sensors with CV < 0.001 (sensors 1, 5, 18, 19). Apply per-regime MinMax scaling; store regime_scalers.
- One-hot encode regime column into six binary dummy columns.

Postcondition: Cleaned, normalised, regime-annotated DataFrame ready for windowing.

UC-02: Train RUL Prediction Model

Actor: ML Engineer

- Partition engine units 80/20 train/validation (unit-based, random_state=42) to prevent temporal data leakage.
- Build sliding windows (size=128, stride=1) producing 3-D NumPy arrays of shape (N, 128, F).
- Train stacked LSTM: LSTM(64, return_sequences=True) → Dropout(0.3) → LSTM(32) → Dropout(0.3) → Dense(1).
- Compile using Adam (lr=0.001), MSE loss; EarlyStopping(patience=1, restore_best_weights=True).
- (Transfer Learning path) Pre-train TS2Vec encoder on X_train for 30 epochs; encode all splits before LSTM training.

UC-03 to UC-08

UC-03 (Predict RUL), UC-04 (Evaluate Model), UC-05 (Apply Regime Clustering), UC-06 (Apply TS2Vec), UC-07 (Visualise PCA & Sensor Trends), and UC-08 (Save/Load Keras Model) follow the same pipeline flow as specified in the original RUL module design.

2.3 Use Case Descriptions — Acoustic Anomaly Detection Module

UC-A01: Preprocess Audio Data

Actor: Data Engineer

Precondition: MIMII dataset audio files available for all four machine types (valves, pumps, fans, slide rails) and selected model IDs (00, 02, 04, 06).

Main Flow:

- Read and normalise audio files using the Librosa library.
- Divide each audio file into 1-second non-overlapping segments.
- Extract MFCC features (40-dimensional) per segment to capture spectral characteristics.
- Extract YAMNet embeddings (1024-dimensional) per segment for high-level acoustic representations.
- Concatenate MFCC and YAMNet features into a single 1064-dimensional feature vector per segment.
- Apply a context window of 5 consecutive segments to incorporate temporal information, producing flattened contextual feature vectors.

Postcondition: Feature matrix of shape (N_segments, 5×1064) ready for autoencoder training.

UC-A02: Train Anomaly Detection Autoencoder

Actor: ML Engineer

Precondition: Preprocessed feature vectors from normal audio files only.

Main Flow:

- Split normal files: 70% train, 15% validation, 15% test.
- Build deep autoencoder — Encoder: 4 FC layers (128 dims each) with BatchNorm, ReLU, Dropout(0.2); Bottleneck: 8-dimensional representation; Decoder: 4 FC layers mirroring the encoder.
- Train exclusively on normal data using MSE reconstruction loss.
- Compute reconstruction error threshold using Method 1 (Mean + Std Dev) or Method 2 (95th Percentile).
- Apply majority vote per file to classify normal vs. anomaly.

Postcondition: Trained autoencoder with selected threshold; classification results per audio file.

UC-A03: Detect Acoustic Anomalies on New Audio

Actor: Maintenance Engineer / System (Automated)

- Receive new audio file input; segment into 1-second windows.
- Extract MFCC (40-D) and YAMNet (1024-D) features; concatenate into 1064-D vectors.
- Apply context windowing (5 segments). Pass feature vectors through trained autoencoder.
- Compute per-segment reconstruction error; apply threshold; aggregate via majority vote.
- Return binary classification: Normal or Anomaly.

UC-A04: Evaluate Anomaly Detection Performance

Actor: ML Engineer

- Compute precision, recall, F1-score, and AUC-ROC for each machine type and model ID.

- Compare statistical threshold vs. percentile-based threshold performance.
- Log all metrics to MLflow with optimizer, batch size, and epoch configurations.

3. Functional & Non-Functional Requirements

3.1 Functional Requirements

RUL Prediction Module

Feature	Description
Data Ingestion	Load space-delimited CMAPSS .txt files with standardised column names (unit_number, time_cycles, op_settings, sensor_1–23).
RUL Label Generation	Compute RUL = max_cycles – current_cycle per engine unit via grouped transform.
Regime Clustering	Cluster op_settings into k=6 regimes using K-Means (n_init=50, random_state=42); validate with silhouette score and elbow analysis.
Sensor Feature Selection	Remove sensors with CV < 0.001 globally and sensors with consistently low per-regime correlation to RUL.
Per-Regime Normalisation	Apply MinMaxScaler [-1, 1] independently per regime to sensor columns.
Time Window Construction	Sliding windows (size=128, step=1) for training; last window per engine for test inference with zero-padding.
LSTM Model Training	Stacked LSTM (64→32→Dense(1)) with Dropout(0.3), Adam (lr=0.001), MSE loss, EarlyStopping on val_RMSE.
Transfer Learning (TS2Vec)	Pre-train TS2Vec encoder (30 epochs, contrastive self-supervised); encode all splits before LSTM training.
Model Evaluation	Compute RMSE, MSE, MAE, MAPE, R ² ; styled comparison table for both model paths.
PCA Visualisation	PCA (2 components) pre- and post-StandardScaler normalisation; regime structure and RUL gradient scatter plots.
Model Persistence	Save trained models in .keras format; verify reloaded models produce identical predictions.

Acoustic Anomaly Detection Module

Feature	Description
Audio Ingestion	Read and normalise MIMII dataset audio files (16 kHz, 16-bit, 8-channel array) using Librosa for four machine types.
Audio Segmentation	Divide each audio file into 1-second non-overlapping segments for uniform feature extraction.
MFCC Extraction	Compute 40-dimensional MFCC features per segment to capture spectral and timbral characteristics.

Feature	Description
YAMNet Embedding Extraction	Extract 1024-dimensional YAMNet embeddings per segment for high-level semantic acoustic representations via transfer learning.
Feature Concatenation	Combine MFCC and YAMNet into a single 1064-dimensional feature vector per segment.
Context Windowing	Apply a 5-segment context window and flatten to incorporate temporal machine behaviour.
Autoencoder Architecture	Encoder (4 FC layers, 128-D, BatchNorm, ReLU, Dropout 0.2) → Bottleneck (8-D) → Decoder (4 FC layers, 128-D).
Normal-Only Training	Train autoencoder exclusively on normal sound data using MSE reconstruction loss.
Threshold Computation	Method 1: Mean Reconstruction Error + 1 Std Dev. Method 2: 95th Percentile of reconstruction errors.
Majority Voting	Aggregate per-segment anomaly labels by majority vote to produce a single file-level classification.
MLflow Tracking	Log optimizer, batch size, epochs, thresholds, and all evaluation metrics for experiment comparison.

3.2 Non-Functional Requirements

Category	Requirement
Performance	RUL inference per engine must complete within 500 ms on CPU. Acoustic anomaly detection per audio file must complete within 2 seconds.
Accuracy (RUL)	Baseline LSTM RMSE on FD004 test set shall not exceed 35 cycles; TS2Vec model targets RMSE < 30 cycles.
Accuracy (Anomaly)	Autoencoder-based detector shall achieve F1-score ≥ 0.80 across all machine types and model IDs.
Reproducibility	All random seeds (random_state=42, n_init=50) fixed; identical results across independent runs.
Scalability	RUL preprocessing pipeline handles all four CMAPSS sub-datasets without code modification; anomaly detector generalises across all four MIMII machine types.
Maintainability	Modular, documented codebase with consistent naming conventions; separate modules for RUL and anomaly detection.
Portability	Python 3.9+ environment; pip-installable dependencies: TensorFlow 2.x, scikit-learn, ts2vec, librosa, pandas, matplotlib, seaborn.
Data Integrity	Test-set scalers are transform-only; unit-based splitting prevents future-cycle data in training windows; audio test data not used in threshold computation.
Deployment	System deployable via FastAPI backend and Gradio interface on Hugging Face Spaces; supports both RUL and acoustic inputs.

4. Software Architecture

4.1 Architecture Style

The system adopts the Pipeline Architecture Design Pattern across both modules. Each stage performs a discrete transformation on its input data and passes structured output to the succeeding stage. Stages are loosely coupled, independently testable, and upgradeable without affecting adjacent stages. The data flow is strictly unidirectional — from raw input ingestion to final prediction output.

4.2 RUL Prediction Pipeline — Component Responsibilities

Stage / Component	Responsibility	Key Functions / Objects
Data Layer	Raw file I/O; loading CMAPSS text files; assigning column headers; dropping null sensor columns (22, 23).	<code>pd.read_csv()</code> , column assignment, <code>drop()</code>
Preprocessing Layer	Computing RUL labels per engine unit; globally scaling operational settings to $[-1, 1]$; test-set scaler application (transform-only).	<code>groupby().transform('max')</code> , <code>MinMaxScaler</code> (fit on train only)
Feature Engineering Layer	K-Means regime detection ($k=6$); CV-based sensor filtering; per-regime Pearson correlation; per-regime sensor normalisation; one-hot regime encoding; PCA visualisation.	<code>KMeans</code> , <code>silhouette_score</code> , <code>MinMaxScaler</code> per regime, <code>PCA</code> , <code>pd.get_dummies()</code>
Windowing Layer	Generating 3-D training/validation arrays via sliding windows; test array extraction via last-window with zero-padding.	<code>create_time_windows(df, window=128, step=1, cols)</code> , <code>create_last_windows(df, window=128, cols)</code>
Model Layer (Baseline)	Stacked LSTM; compilation and training with <code>EarlyStopping</code> and <code>ModelCheckpoint</code> ; serialisation to .keras format.	<code>Sequential([LSTM(64), Dropout, LSTM(32), Dropout, Dense(1)])</code> , <code>Adam(lr=0.001)</code>
Model Layer (TS2Vec)	Self-supervised pre-training of <code>TS2Vec</code> encoder; encoding all splits into temporal embeddings; LSTM trained on embeddings.	<code>TS2Vec(input_dims=F)</code> , <code>.fit(X_train, n_epochs=30)</code> , <code>.encode(X)</code>
Evaluation Layer	Computing RMSE, MSE, MAE, MAPE, R^2 for both paths; plotting loss and RMSE curves; styled comparison <code>DataFrame</code> .	<code>compute_regression_metrics()</code> , <code>evaluate()</code> , <code>plot_loss()</code> , <code>plot_RMSE()</code>

4.3 Acoustic Anomaly Detection Pipeline — Component Responsibilities

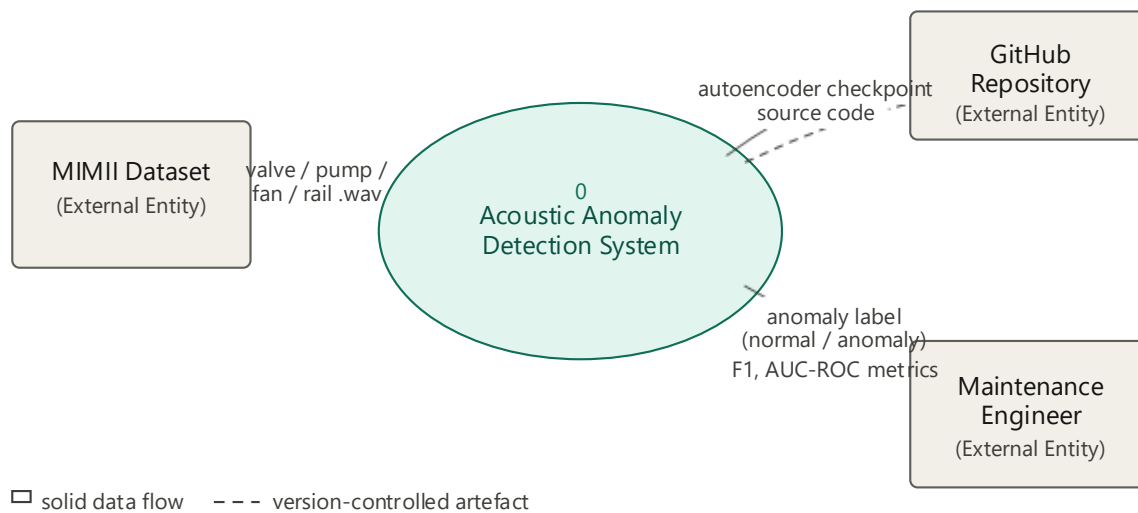
Stage / Component	Responsibility	Key Functions / Objects
Audio Data Layer	Loading MIMII audio files (.wav); applying Librosa normalisation; organising by machine type and model ID.	librosa.load(), librosa.util.normalize()
Segmentation Layer	Dividing audio files into 1-second non-overlapping segments for uniform feature extraction.	librosa frame slicing, np.split()
Feature Extraction Layer	Computing 40-D MFCC features per segment; extracting 1024-D YAMNet embeddings per segment; concatenating into 1064-D vectors.	librosa.feature.mfcc(), yamnet_model(segment)
Context Windowing Layer	Applying 5-segment context windows and flattening to produce final feature vectors of size $5 \times 1064 = 5320$.	np.concatenate(), custom windowing logic
Autoencoder Model Layer	Building, training, and saving the deep autoencoder (Encoder $4 \times 128 \rightarrow$ Bottleneck 8 $\rightarrow$ Decoder 4×128) on normal data only.	torch.nn or tf.keras Sequential, MSELoss, Adam/RMSprop
Threshold & Voting Layer	Computing anomaly threshold (statistical or percentile-based); applying per-segment classification; majority vote per file.	np.percentile(), custom majority_vote()
Evaluation Layer	Computing precision, recall, F1-score, AUC-ROC per machine and model ID; logging to MLflow.	sklearn.metrics, mlflow.log_metric()

5. Data Flow & System Behaviour

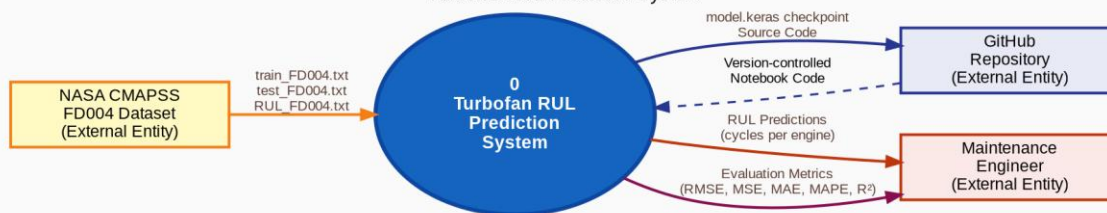
5.1 Context-Level Data Flow (Level 0)

At the highest level of abstraction, the system has two external data sources and two categories of output consumers. The NASA CMAPSS FD004 dataset provides engine sensor time-series data. The MIMII dataset provides machine audio recordings. The system produces RUL predictions and anomaly classifications directed to the Maintenance Engineer, and serialised model checkpoints directed to the GitHub repository.

2. DATA FLOW DIAGRAM — LEVEL 0 (Context Diagram)
Acoustic Anomaly Detection System



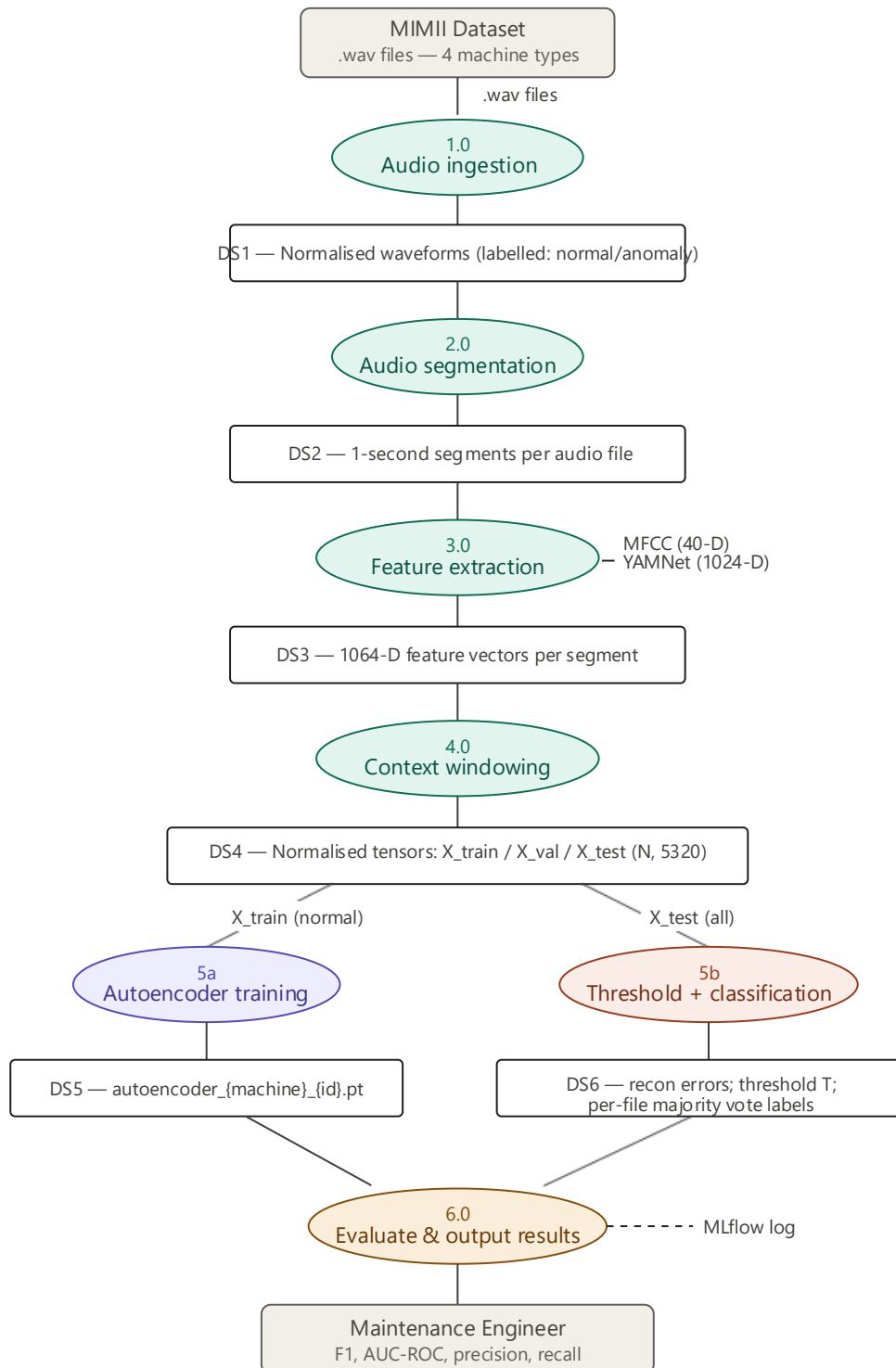
2. DATA FLOW DIAGRAM — LEVEL 0 (Context Diagram)
Turbofan RUL Prediction System



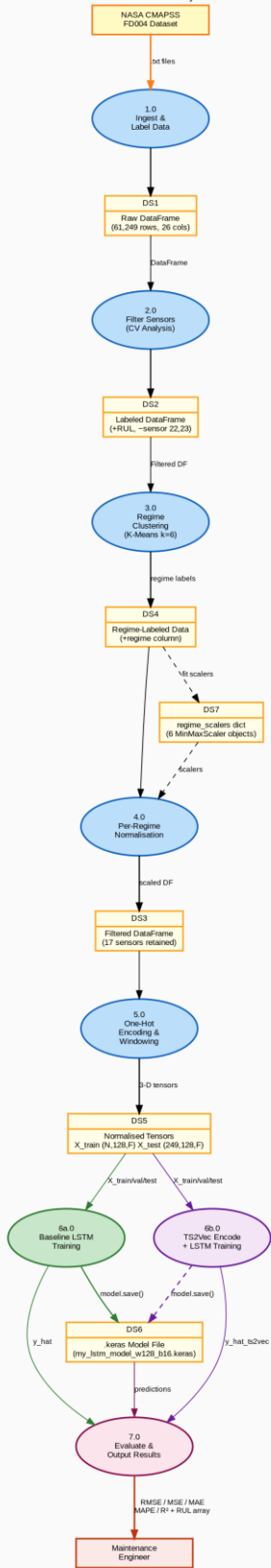
5.2 RUL Module — Detailed Data Flow (Level 1)

Process	Input	Output
Ingest & Label	CMAPSS .txt files	DataFrame: 26 columns + RUL target; 61,249 rows; sensors 22 & 23 dropped.
Sensor Filtering (CV)	Labelled DataFrame	Reduced DataFrame: sensors 1, 5, 18, 19 removed (CV < 0.001).
Regime Detection	Op settings (op1–op3)	K-Means k=6 cluster labels appended as 'regime' column.
Per-Regime Normalisation	Regime-labelled sensor data	Sensor columns scaled to $[-1, 1]$ per regime; regime_scalers dictionary stored.
One-Hot Encoding	Regime column	Six binary dummy columns (regime_0...regime_5) replace categorical regime.
Unit Train/Val Split	Final feature matrix	80% train / 20% validation units; unit-based split prevents leakage.
Time Window Creation	Train/val/test DataFrames	3-D NumPy arrays: X_train (N, 128, F), X_val (M, 128, F), X_test (249, 128, F).
Model Training	X_train, y_train, X_val, y_val	Trained LSTM weights; training history (loss and RMSE per epoch).
Evaluation & Output	X_test, y_test from RUL_FD004	RMSE, MSE, MAE, MAPE, R ² metrics; styled DataFrame; saved .keras model.

5.3 Acoustic Anomaly Detection Module — Detailed Data Flow (Level 1)

3. DATA FLOW DIAGRAM — LEVEL 1 (Detailed Pipeline)
Acoustic Anomaly Detection System

3. DATA FLOW DIAGRAM — LEVEL 1 (Detailed Pipeline)
Turbopan RUL Prediction System



Process	Input	Output
Audio Ingestion	MIMII .wav files (4 machine types, IDs 00, 02, 04, 06)	Normalised waveform arrays per file; labelled normal / anomaly.
Audio Segmentation	Normalised waveform arrays	1-second segments per audio file (N_segs per file).
MFCC Extraction	Audio segments	40-D MFCC feature vector per segment.
YAMNet Embedding Extraction	Audio segments	1024-D YAMNet embedding per segment.
Feature Concatenation	MFCC (40-D) + YAMNet (1024-D)	1064-D feature vector per segment.
Context Windowing	1064-D vectors (consecutive)	5320-D flattened context vector per window (5×1064).
Normal/Anomaly Split	Labelled feature vectors	Normal: 70% train, 15% val, 15% test; all anomaly data reserved for test.
Autoencoder Training	Normal feature vectors (train split)	Trained autoencoder weights; reconstruction errors on validation set.
Threshold Computation	Reconstruction errors (train/val)	Scalar threshold T (statistical or percentile-based).
Anomaly Classification	Test feature vectors + threshold T	Per-segment binary labels; file-level label via majority vote.
Evaluation	Predicted labels + ground-truth labels	Precision, recall, F1-score, AUC-ROC per machine type and model ID.

5.4 Sequence Description — Anomaly Detection Training Flow

- The ML Engineer invokes the audio loader, which reads and normalises MIMII audio files, organised by machine type and model ID.
- The segmentation module divides each file into 1-second non-overlapping segments.
- The feature extraction module computes 40-D MFCC and 1024-D YAMNet embeddings per segment, concatenating them into 1064-D vectors.
- The context windowing module applies a 5-segment window and flattens to produce 5320-D input vectors.
- Normal data is split 70/15/15 (train/val/test). Anomalous data is held exclusively for testing.
- The autoencoder is compiled with MSE loss and Adam or RMSprop optimizer; training runs for 50–100 epochs.
- Reconstruction errors on the validation set are used to determine the anomaly threshold (statistical or percentile-based).
- At inference, per-segment reconstruction errors are compared against the threshold; majority vote per file produces the final classification.

5.5 State Transitions — Audio File Lifecycle

- Raw: Audio file loaded from MIMII dataset, unnormalised.
- Normalised: Waveform amplitude normalised using Librosa.
- Segmented: Divided into 1-second non-overlapping windows.
- Feature-Extracted: MFCC (40-D) and YAMNet (1024-D) computed per segment; concatenated to 1064-D.
- Context-Windowed: Five consecutive feature vectors flattened to 5320-D input tensor.
- Reconstructed: Input fed through trained autoencoder; reconstruction error computed (MSE).
- Classified: Per-segment label assigned (normal / anomaly) based on threshold; file-level label determined by majority vote.
- Evaluated: File-level label compared against ground truth; contribution to aggregate F1 and AUC-ROC computed.

6. Data Design & Storage

6.1 Data Sources

Data Store	Format	Content
train_FD004.txt	Space-delimited .txt	Training engine sensor readings — 26 columns, ~61,249 rows, 248 engine units with full run-to-failure histories.
test_FD004.txt	Space-delimited .txt	Test engine sensor windows — 249 engine units, variable cycles per unit; only partial histories observed.
RUL_FD004.txt	Single-column .txt	True RUL at the last observed cycle for each of the 249 test engines — ground-truth regression labels.
MIMII Dataset (.wav)	16 kHz WAV, 8-channel	Normal and anomalous machine sounds for 4 machine types (valve, pump, fan, slide rail); 7 models each; IDs 00, 02, 04, 06 used.
my_lstm_model_w128_b16.keras	Keras SavedModel	Serialized LSTM weights and architecture for RUL inference and deployment.
autoencoder_{machine}_{id}.pt / .h5	PyTorch / Keras checkpoint	Trained autoencoder weights per machine type and model ID.

6.2 Feature Schema — RUL Module (Post-Preprocessing)

Column Group	Count	Notes
op_setting_1, op_setting_2, op_setting_3	3	Globally scaled to [-1, 1]; used for K-Means regime assignment; included as model input features.
Retained sensor columns	~17	sensor_2, 3, 4, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 20, 21 — retained after CV and per-regime correlation filtering.
regime_0 ... regime_5	6	One-hot encoded K-Means cluster membership; binary integer columns appended after regime assignment.
RUL (target)	1	Regression target in cycles; not included in input tensors; used only as y_train, y_val, y_test.
unit_number, time_cycles	2	Engine identifier and cycle counter; excluded from feature tensors; used for grouping, splitting, and windowing.

6.3 Feature Schema — Acoustic Anomaly Detection Module (Post-Preprocessing)

Column Group	Dimension	Notes
MFCC features	40	Mel Frequency Cepstral Coefficients per 1-second audio segment; captures spectral envelope and timbral characteristics.
YAMNet embeddings	1024	High-level acoustic representations from the pre-trained YAMNet model; provides semantic sound classification features.
Concatenated feature vector	1064	MFCC (40-D) + YAMNet (1024-D) combined per segment; input to context windowing.
Context window vector	5320	Five consecutive 1064-D vectors flattened; final input to the autoencoder (5×1064).
Reconstruction error	1 (scalar)	MSE between autoencoder input and output; primary anomaly score per segment.
Anomaly label (segment)	1 (binary)	0 = Normal, 1 = Anomaly; derived by comparing reconstruction error against threshold T.
File-level label	1 (binary)	0 = Normal, 1 = Anomaly; derived by majority vote across all segments of the file.

7. Technology Stack

Layer	Technology	Usage in This Project
Language	Python 3.9+	Primary implementation language for all pipeline stages across both modules.
Data	pandas	DataFrame manipulation, grouped RUL computation, feature selection, and regime label merging.
Numerics	NumPy	3-D tensor construction for LSTM input; zero-padding logic; reconstruction error computation.
Machine Learning	scikit-learn	KMeans clustering, MinMaxScaler, StandardScaler, PCA, train_test_split, regression metrics.
Deep Learning (RUL)	TensorFlow / Keras	LSTM, Dense, Dropout layers; Adam optimizer; EarlyStopping; ModelCheckpoint; model serialisation.
Transfer Learning (RUL)	TS2Vec	Self-supervised contrastive time-series representation learning; 30-epoch pre-training; encode() for all data splits.
Audio Processing	Librosa	Audio file loading, normalisation, segmentation, and MFCC feature extraction.
Audio Transfer Learning	YAMNet (TF Hub)	Pre-trained audio classification network; extracts 1024-D semantic embeddings from audio segments.
Deep Learning (Anomaly)	TensorFlow / Keras or PyTorch	Deep autoencoder (FC layers with BatchNorm, ReLU, Dropout); MSE loss; Adam / RMSprop optimizers.
Experiment Tracking	MLflow	Logging metrics (RMSE, R ² , F1, AUC-ROC), hyperparameters, model artifacts; centralized experiment comparison.
API Backend	FastAPI	REST API endpoint for receiving sensor windows and audio files; applies preprocessing pipeline; returns predictions.
UI Interface	Gradio	Interactive web interface for operational settings, sensor values, and audio file input; displays RUL and anomaly results.
Deployment	Hugging Face Spaces	Cloud hosting for Gradio interface; real-time RUL and anomaly detection accessible via web browser.
Visualisation	matplotlib / seaborn	Training loss curves, RMSE curves, PCA scatter plots, sensor degradation plots, spectrogram visualisations.
Environment	Jupyter Notebook / JupyterLab	Interactive development, experiment logging, and inline visualisation.
Version Control	Git / GitHub	Source code repository with commit history, branching strategy, and README documentation.

8. Logical Class / Module Diagram

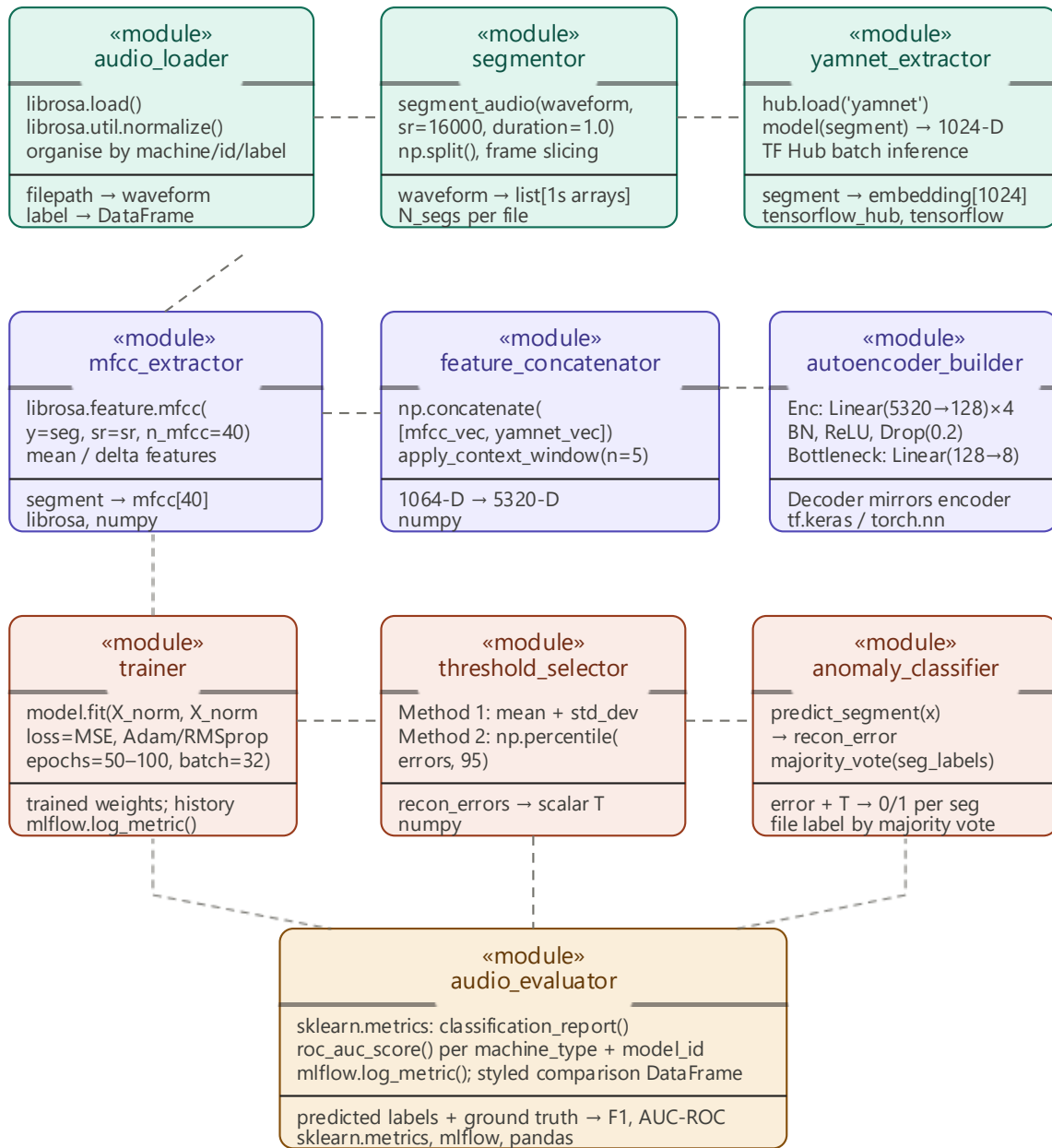
The implementation follows a functional, notebook-based pipeline rather than an object-oriented class hierarchy. The following tables illustrate the logical module decomposition across both modules.

8.1 RUL Prediction Module

Module	Key Functions / Objects	Dependencies
data_loader	pd.read_csv(), column assignment list, drop(['sensor_22','sensor_23'])	pandas
rul_calculator	groupby('unit_number')['time_cycles'].transform('max'), RUL = max – current	pandas, data_loader
regime_detector	KMeans(n_clusters=6, n_init=50), silhouette_score(), elbow inertia plot, kmeans.predict()	sklearn.cluster, sklearn.metrics, matplotlib, data_loader
feature_selector	CV = std / mean , threshold filter (CV < 0.001), per-regime corr()['RUL'], drop(low_corr_sensors)	pandas, regime_detector
normalizer	MinMaxScaler(feature_range=(-1,1)) per regime, regime_scalers dict, transform-only on test	sklearn.preprocessing, regime_detector
pca_visualiser	StandardScaler(), PCA(n_components=2), scatter plot coloured by RUL per regime	sklearn.decomposition, matplotlib, normalizer
windowing	create_time_windows(df, window_size, step, cols) → (X, y); create_last_windows(df, window_size, cols) → X with zero-pad	numpy, pandas
model_builder	Sequential([LSTM(64), Dropout(0.3), LSTM(32), Dropout(0.3), Dense(1)]), model.compile(), EarlyStopping(patience=1), model.save()	tensorflow.keras, windowing
ts2vec_encoder	TS2Vec(input_dims=F), .fit(X_train, n_epochs=30), .encode(X) for all splits	ts2vec, windowing
evaluator	compute_regression_metrics() → RMSE, MSE, MAE, MAPE, R ² ; plot_loss(); plot_RMSE()	sklearn.metrics, numpy, matplotlib, pandas

8.2 Acoustic Anomaly Detection Module

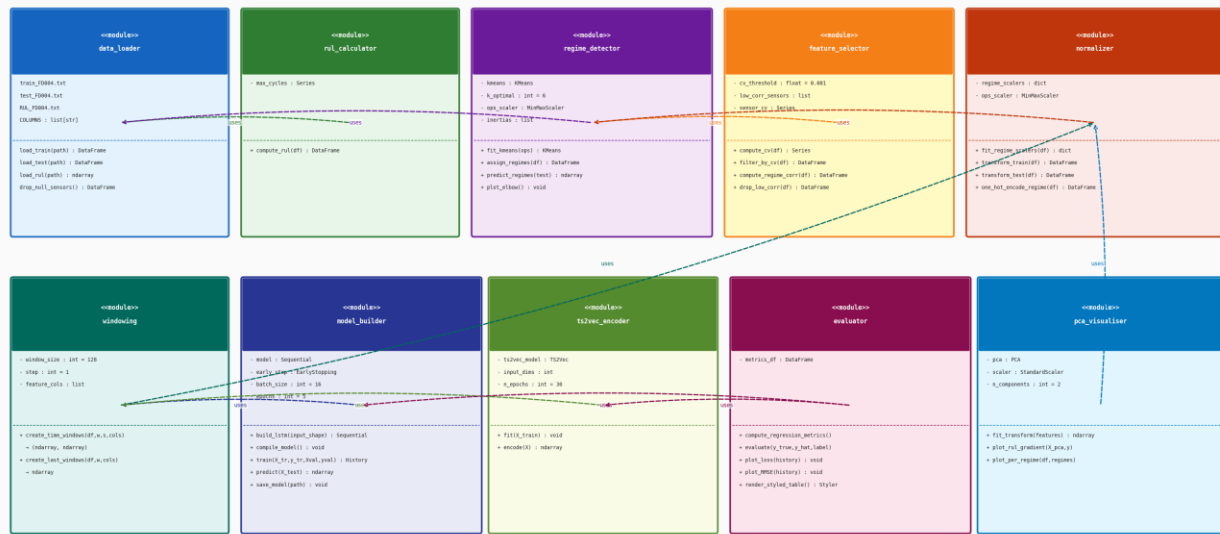
4. CLASS / MODULE DIAGRAM
Acoustic Anomaly Detection System — Logical Module View



----- dependency / uses audio I/O & preprocessing feature engineering
 training & detection evaluation

4. CLASS / MODULE DIAGRAM

Turbofan RUL Prediction System — Logical Module View



Module	Key Functions / Objects	Dependencies
audio_loader	librosa.load(filepath, sr=16000), librosa.util.normalize(); organises files by machine_type/model_id/label	librosa, os, pathlib
segmentor	segment_audio(waveform, sr=16000, duration=1.0) → list of 1-second arrays; non- overlapping split	numpy, librosa
mfcc_extractor	librosa.feature.mfcc(y=segment, sr=sr, n_mfcc=40); mean/delta features → 40-D vector per segment	librosa, numpy
yamnet_extractor	hub.load('yamnet'); model(segment) → embeddings (1024-D per segment)	tensorflow_hub, tensorflow
feature_concatenator	np.concatenate([mfcc_vec, yamnet_vec], axis=-1) → 1064-D; apply_context_window(features, n=5) → 5320-D	numpy
autoencoder_builder	Encoder: Linear(5320→128)×4, BN, ReLU, Dropout(0.2); Bottleneck: Linear(128→8); Decoder: Linear(8→128)×4 → Linear(128→5320)	tensorflow.keras or torch.nn

Module	Key Functions / Objects	Dependencies
trainer	model.fit(X_normal_train, X_normal_train, loss=MSE, optimizer=Adam/RMSprop, epochs=50–100, batch=32–128)	tensorflow.keras / torch, mlflow
threshold_selector	Method1: mean_error + std_error; Method2: np.percentile(errors, 95); returns scalar T	numpy
anomaly_classifier	predict_segment(x) → recon_error; classify(error, T) → 0/1; majority_vote(segment_labels) → file_label	numpy
audio_evaluator	sklearn.metrics: classification_report(), roc_auc_score(); mlflow.log_metric(); per machine_type + model_id	sklearn.metrics, mlflow, pandas

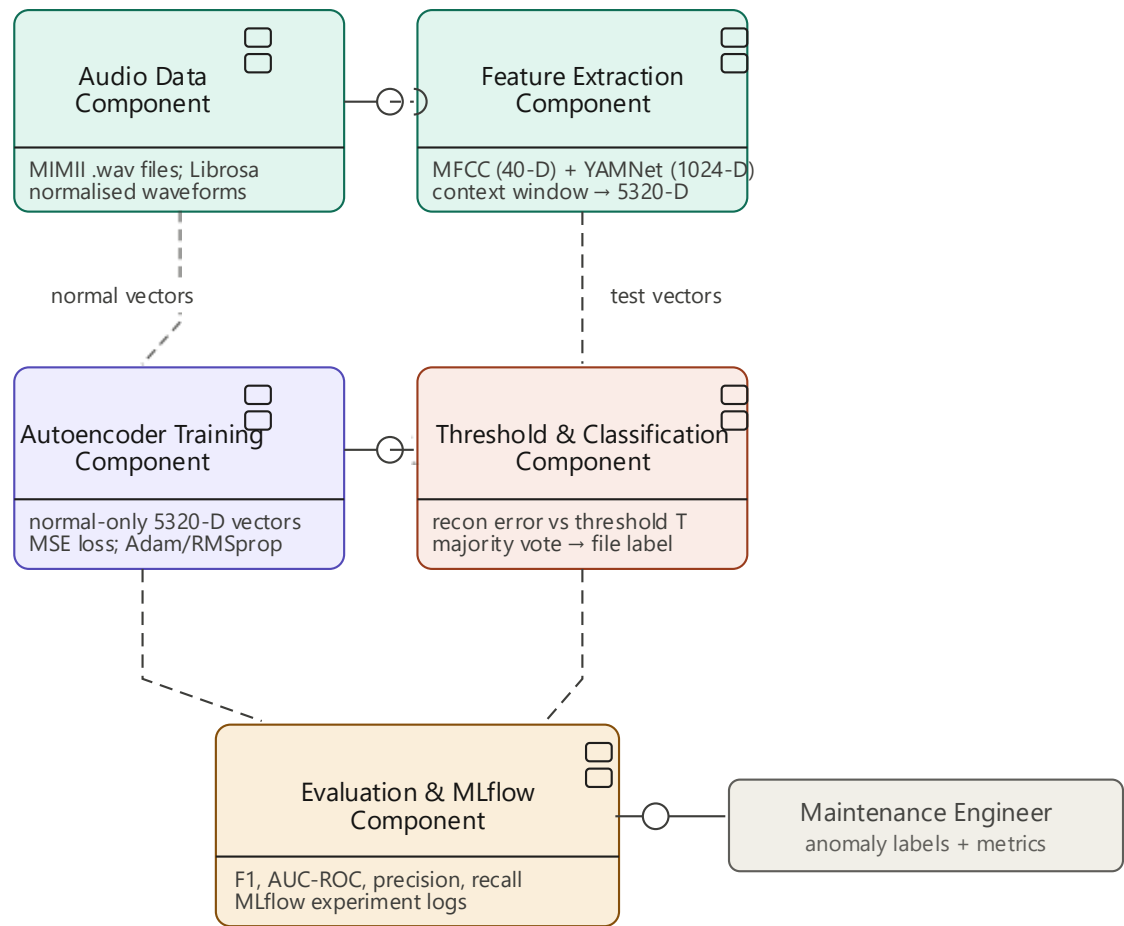
9. Component Diagram

9.1 RUL Prediction — High-Level Component Overview

Component	Provided Interface	Required Interface / Dependency
Data & Preprocessing Component	Structured, regime-labelled, normalised DataFrame ready for windowing	NASA CMAPSS flat files (train/test/RUL.txt); file system access
Feature Engineering Component	Filtered and per-regime-normalised feature matrix; regime_scalers dict; one-hot regime columns	Preprocessed DataFrame from Data Component; scikit-learn KMeans, MinMaxScaler, PCA
Windowing Component	3-D NumPy tensors: X_train, X_val, X_test of shape (N, 128, F) with labels y_train, y_val	Normalised DataFrame from Feature Engineering Component; NumPy
Model Training Component	Trained LSTM weights (.keras checkpoint); training history (loss and RMSE curves)	Windowed tensors from Windowing Component; TensorFlow/Keras; TS2Vec (optional)
Evaluation Component	Metric report (RMSE, MSE, MAE, MAPE, R ²); styled comparison DataFrame; diagnostic plots	Predictions from Model Training; ground-truth labels (RUL_FD004); matplotlib, seaborn, pandas

9.2 Acoustic Anomaly Detection — High-Level Component Overview

5. COMPONENT / PIPELINE DIAGRAM
Acoustic Anomaly Detection System



Deployment: FastAPI (POST /predict/anomaly) → Gradio UI → Hugging Face Spaces · MLflow experiment tracking



Component	Provided Interface	Required Interface / Dependency
Audio Data Component	Normalised waveform arrays per file; organised by machine type, model ID, and label	MIMI dataset .wav files; Librosa; file system access
Feature Extraction Component	1064-D feature vectors per segment; context-windowed 5320-D input vectors	Normalised waveforms from Audio Data Component; Librosa (MFCC), YAMNet (TF Hub)

Component	Provided Interface	Required Interface / Dependency
Autoencoder Training Component	Trained autoencoder weights; reconstruction error distribution on validation data	Normal-only 5320-D feature vectors from Feature Extraction Component; TensorFlow/Keras or PyTorch
Threshold & Classification Component	Per-segment binary labels; file-level binary labels via majority vote	Reconstruction errors from Autoencoder Component; threshold T from Method 1 or Method 2
Evaluation & MLflow Component	Precision, recall, F1-score, AUC-ROC per machine and model ID; MLflow experiment logs	File-level labels from Classification Component; ground-truth labels; MLflow

9.3 Integrated Deployment Component Overview

Component	Provided Interface	Required Interface / Dependency
FastAPI Backend (api.py)	POST /predict/rul — accepts sensor window JSON; POST /predict/anomaly — accepts audio file	Trained LSTM model (.keras); trained autoencoder checkpoint; preprocessing scalers; Librosa; YAMNet
Gradio UI (gradio_ui.py)	Interactive web interface for operational settings, sensor values, and audio file upload; displays RUL estimate and anomaly label	FastAPI endpoints; Gradio library
Hugging Face Spaces	Public web URL for accessing Gradio interface; real-time inference	Gradio UI; model checkpoints stored in Spaces repository

10. Deployment Diagram

10.1 Development Environment

- Platform: Python 3.9 or above, running on Jupyter Notebook or JupyterLab.
- Required libraries: `pip install tensorflow scikit-learn pandas matplotlib seaborn ts2vec librosa tensorflow-hub jinja2 fastapi gradio mlflow`.
- Optional GPU support: CUDA 11.x with cuDNN 8.x for TensorFlow GPU acceleration; full CPU-only execution supported.
- Minimum recommended hardware: 16 GB RAM; 4-core CPU; GPU strongly recommended for TS2Vec pre-training and YAMNet batch feature extraction.
- Storage requirement: approximately 2 GB (CMAPSS data files, MIMII audio files, and serialised model checkpoints).

10.2 Node and Artefact Distribution

Deployment Node	Hosted Artefact	Description
Developer Workstation / Colab	Turbofan_RUL.ipynb, Acoustic_Anomaly.ipynb	Main development notebooks; all pipeline code in modular, commented cells.
File System / Cloud Storage	/data/ (CMAPSS .txt files, MIMII .wav files)	Raw input data; excluded from version control via .gitignore.
File System / Cloud Storage	/models/ (.keras and .pt/.h5 checkpoints)	Serialised model weights; versioned by window size, batch size, and machine type.
GitHub Repository	Source code, docs, README, requirements.txt	Full project repository with branching strategy and commit history.
Hugging Face Spaces	Gradio interface + FastAPI backend + model weights	Public deployment for RUL prediction; accessible via URL with real-time inference.
REST API Server	FastAPI (api.py)	POST /predict/rul and POST /predict/anomaly endpoints; JSON and audio file inputs.
Cloud Platform (optional)	GCP Vertex AI / AWS SageMaker	Managed endpoints with auto-scaling; versioned model registry for production.
Edge Device (optional)	TFLite models (.tflite)	Quantised LSTM and autoencoder for on-device inference on resource-constrained industrial hardware.

10.3 Model Serving Options

Option	Technology	Notes
REST API	FastAPI + TF Serving	POST /predict/rul with JSON sensor window; POST /predict/anomaly with audio file → binary label. Real-time alerting.
Interactive Web UI	Gradio + Hugging Face Spaces	User-facing interface; accepts operational settings, sensor values, and audio upload; displays predictions.
Batch Inference	Python script + cron	Nightly batch scoring of all active engine units and audio recordings; writes results to a reporting table.
Embedded (Edge)	TFLite conversion	Convert .keras models to .tflite for on-device inference; reduced latency and no network dependency.
Cloud Managed	GCP Vertex AI / AWS SageMaker	Managed endpoints with auto-scaling, versioned registry, and A/B testing for model comparison.
Experiment Tracking	MLflow	Centralised logging of all hyperparameter configurations, metrics (RMSE, F1, AUC-ROC), and model artifacts.

11. Activity Diagram — End-to-End Workflow

11.1 RUL Prediction Pipeline (Phases 1–6)

Phase 1 — Data Ingestion & Labelling

- Load train_FD004.txt, test_FD004.txt, and RUL_FD004.txt from the working directory.
- Assign column names: unit_number, time_cycles, op_setting_1–3, sensor_1–23. Drop sensor_22 and sensor_23.
- Compute RUL labels using max_cycles_per_unit – current_cycle (grouped transform).

Phase 2 — Preprocessing & Feature Engineering

- MinMax scale operational settings to $[-1, 1]$ globally (fit on training; transform-only on test).
- Determine optimal k via elbow and silhouette analysis (k=6 selected). Fit K-Means; assign regimes.
- Compute CV per sensor; discard sensors with CV below 0.001. Compute per-regime Pearson correlation; discard sensors 1, 5, 18, 19.
- Apply per-regime MinMax scaling $[-1, 1]$; store regime_scalers. One-hot encode the regime column.

Phase 3 — Exploratory Visualisation

- Apply PCA (2 components) before and after StandardScaler normalisation; observe degradation curve and regime separability.
- Plot per-regime sensor degradation curves for selected sensors across varying engines.

Phase 4 — Windowing

- Apply unit-based 80/20 train/validation split (random_state=42).
- create_time_windows(train_df, 128, 1, cols) → X_train (N, 128, F), y_train.
- create_last_windows(test_df, 128, cols) → X_test (249, 128, F); zero-pad short engine histories.

Phase 5A — Baseline LSTM Training

- Build Sequential: LSTM(64, return_sequences=True) → Dropout(0.3) → LSTM(32) → Dropout(0.3) → Dense(1).
- Compile: Adam(lr=0.001), loss='mse'. Fit ≤5 epochs with EarlyStopping(patience=1, restore_best_weights=True).

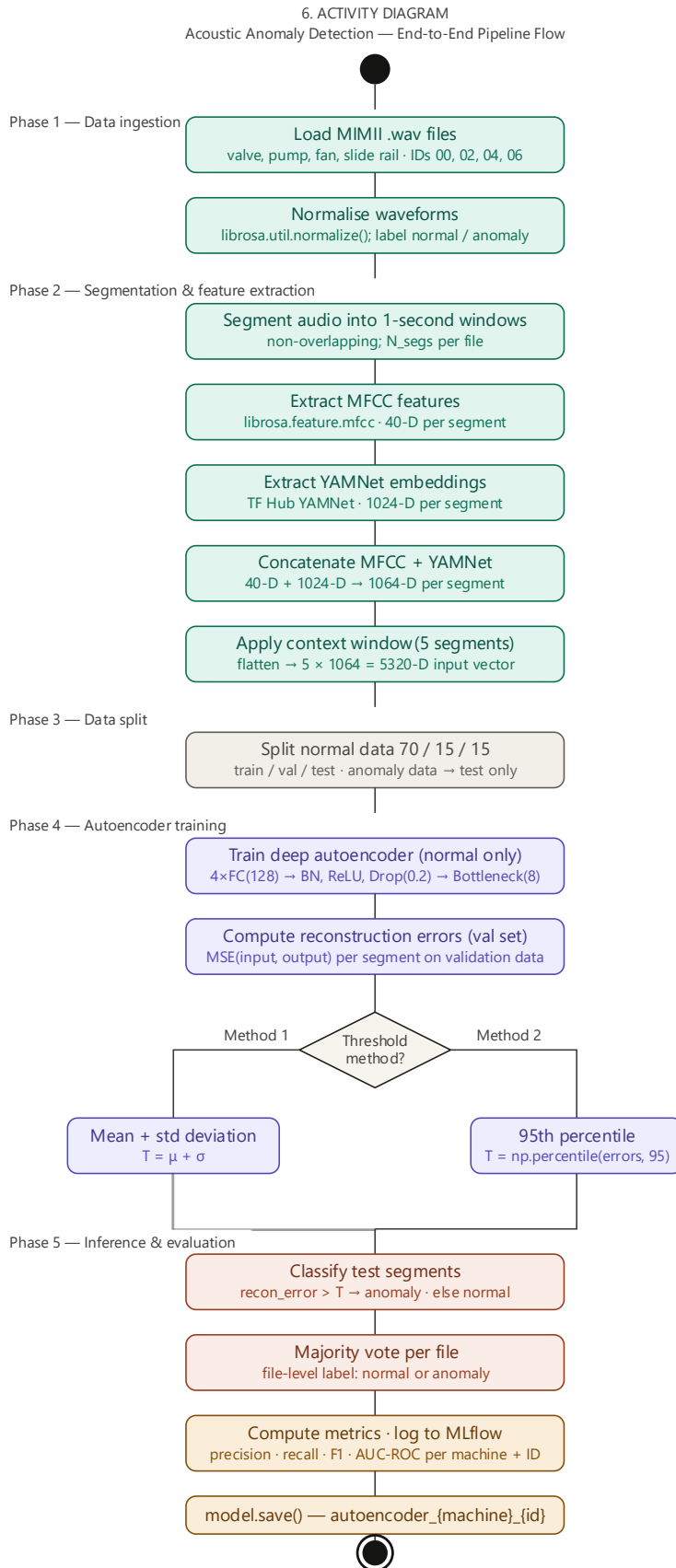
Phase 5B — Transfer Learning (TS2Vec) Training

- Instantiate TS2Vec(input_dims=F); pre-train on X_train for 30 epochs.
- Encode X_train, X_val, X_test → embeddings. Build identical LSTM on embedding input; compile and fit ≤6 epochs.

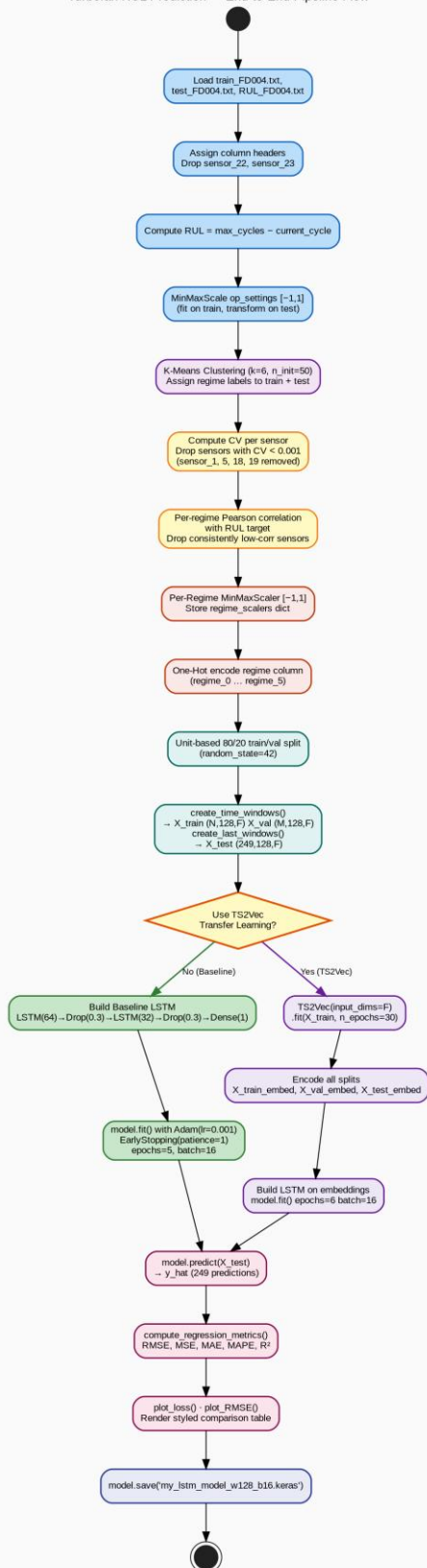
Phase 6 — Evaluation & Persistence

- Compute RMSE, MSE, MAE, MAPE, R^2 for both model paths. Render styled comparison DataFrame.
- Save baseline model as `my_lstm_model_w128_b16.keras` to `/models/`.

11.2 Acoustic Anomaly Detection Pipeline (Phases A1–A5)



5. ACTIVITY DIAGRAM
Turbofan RUL Prediction — End-to-End Pipeline Flow



Phase A1 — Audio Data Ingestion

- Load MIMII .wav files for all four machine types (valve, pump, fan, slide rail) using model IDs 00, 02, 04, 06.
- Normalise waveform amplitude using Librosa; label each file as normal or anomalous based on directory structure.
- Visualise waveform plots, spectrograms, Log-Mel spectrograms, MFCC patterns, and PCA projections for exploratory analysis.

Phase A2 — Feature Extraction & Preprocessing

- Divide each audio file into 1-second non-overlapping segments.
- Extract 40-D MFCC features and 1024-D YAMNet embeddings per segment.
- Concatenate into 1064-D feature vectors. Apply 5-segment context window; flatten to 5320-D.

Phase A3 — Data Splitting

- Split normal files: 70% train, 15% validation, 15% test. All anomalous files reserved for test evaluation only.

Phase A4 — Autoencoder Training & Threshold Computation

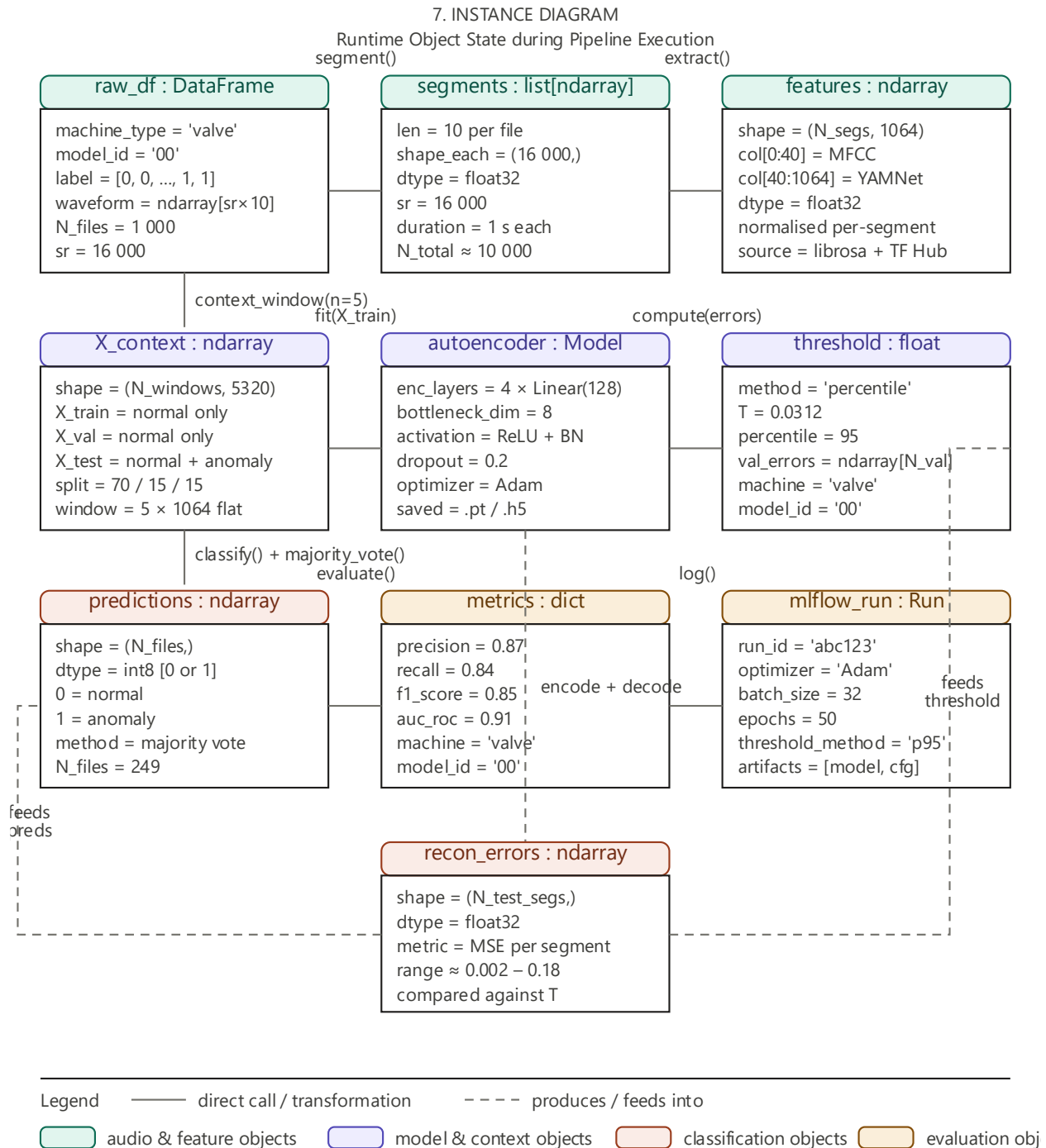
- Build deep autoencoder: Encoder (4×FC 128, BN, ReLU, Dropout 0.2) → Bottleneck (8-D) → Decoder (4×FC 128).
- Train on normal training data using MSE loss; optimizers: Adam or RMSprop; epochs: 50–100; batch: 32–128.
- Compute anomaly threshold from validation reconstruction errors using Method 1 (Mean + Std Dev) or Method 2 (95th Percentile).

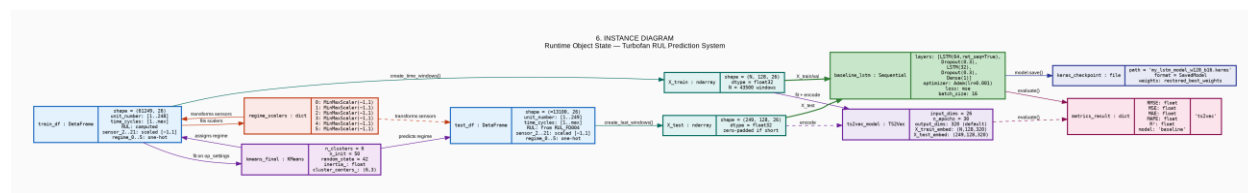
Phase A5 — Evaluation & MLflow Logging

- Apply trained autoencoder to test data (normal and anomalous). Compare per-segment reconstruction errors against threshold T.
- Apply majority vote per file to produce file-level binary labels.
- Compute precision, recall, F1-score, AUC-ROC per machine type and model ID.
- Log all hyperparameters (optimizer, batch size, epochs, threshold method) and metrics to MLflow.

12. Instance Diagram — Runtime Object State

The instance diagrams below capture the runtime state of all principal objects during a complete execution of the pipeline, from raw file loading through to model prediction and evaluation. They illustrate the concrete shapes, types, and inter-object relationships that exist in memory at the end of a successful training run.





13. Testing & Validation Plan

13.1 RUL Module — Test Cases

Test Case	Expected Result	Validation Method
Data loading integrity	DataFrame shape (61249, 26); no NaN in retained columns.	<code>data.isnull().sum() == 0</code> ; <code>data.shape</code> assertion.
RUL label correctness	$RUL \geq 0$ for all rows; $RUL = 0$ at last observed cycle per unit.	<code>assert (data['RUL'] >= 0).all()</code> ; check last-row RUL per unit.
K-Means regime validity	Silhouette score > 0.5 for $k=6$; no empty clusters.	Print silhouette scores; <code>data['regime'].value_counts()</code> .
No data leakage	Test-set scalers use transform-only; no test engine units in training split.	Verify <code>ops_scaler.transform()</code> on test; assert no overlap of unit IDs.
Window shape correctness	X_{train} shape (N, 128, F); X_{test} shape (249, 128, F).	<code>print(X_train.shape)</code> ; assert <code>X_test.shape[0] == 249</code> .
Zero-padding correctness	Engines with < 128 cycles receive zero-padded windows; output shape (128, F).	Inspect <code>create_last_windows()</code> for known short engines.
Baseline LSTM convergence	<code>val_RMSE</code> decreases over epochs; EarlyStopping triggers within 5 epochs.	Plot <code>history['val_root_mean_squared_error']</code> ; confirm improvement.
Baseline RMSE threshold	Test RMSE < 40 cycles on FD004 test set.	<code>evaluate(y_test, y_hat_test)</code> ; <code>compute_regression_metrics()</code> .
TS2Vec improvement	TS2Vec-LSTM RMSE $\leq$ Baseline RMSE; R^2 improvement observable.	Compare model performance DataFrames from both paths.
Model serialisation	Reloaded .keras model produces predictions identical to in-session predictions.	<code>tf.keras.models.load_model()</code> ; <code>np.allclose(preds_session, preds_loaded)</code> .

13.2 Acoustic Anomaly Detection Module — Test Cases

Test Case	Expected Result	Validation Method
Audio loading integrity	All audio files loaded without error; no NaN/Inf in normalised waveforms.	<code>np.isnan(waveform).any() == False;</code> <code>np.isinf(waveform).any() == False.</code>
Segmentation correctness	Each 10-second file produces 10 non-overlapping 1-second segments; no overlap.	<code>Assert len(segments) == floor(duration);</code> inspect segment boundaries.
MFCC dimension	Each segment produces a 40-D MFCC vector.	<code>Assert mfcc_vec.shape == (40,).</code>
YAMNet dimension	Each segment produces a 1024-D YAMNet embedding.	<code>Assert yamnet_emb.shape == (1024,).</code>
Context window dimension	Each context window produces a 5320-D feature vector (5 × 1064).	<code>Assert context_vec.shape == (5320,).</code>
Normal-only training	Autoencoder training data contains no anomalous samples.	<code>Assert all labels in training set == 0.</code>
Threshold validity	Threshold $T > 0$; Method 2 (95th percentile) $\geq$ Method 1 (mean + std) in most cases.	<code>Print T for both methods; verify $T > 0$.</code>
Majority vote consistency	File-level label consistent with dominant per-segment label.	Manual inspection of 5 files per machine type.
F1-score threshold	F1-score ≥ 0.80 across all machine types and model IDs.	<code>classification_report(y_true, y_pred);</code> per machine type breakdown.
MLflow logging	All metrics and parameters logged to MLflow for each experiment run.	Inspect MLflow UI; assert all required keys present in <code>run.data.metrics</code> .
Autoencoder serialisation	Reloaded autoencoder produces reconstruction errors within float tolerance of in-session errors.	<code>np.allclose(errors_session, errors_loaded, atol=1e-5).</code>

14. MLflow Experiment Tracking

MLflow is used to manage and track all ML experiments across both modules. It provides a centralised interface for analysing experiments and comparing results, ensuring reproducibility and facilitating efficient model selection.

14.1 Tracked Parameters and Metrics

Module	Logged Parameters & Metrics
RUL Prediction (Baseline LSTM)	window_size, batch_size, epochs, learning_rate, dropout_rate; RMSE, MSE, MAE, MAPE, R ² (train, val, test)
RUL Prediction (TS2Vec LSTM)	ts2vec_epochs, window_size, batch_size; RMSE, MSE, MAE, MAPE, R ² (train, val, test); embedding_dim
Acoustic Anomaly Detection	optimizer (Adam/RMSprop), batch_size (32/64/128), epochs (50–100), threshold_method; precision, recall, F1-score, AUC-ROC per machine_type and model_id

15. Challenges & Future Work

15.1 Challenges

RUL Prediction — Complexity of FD004

The FD004 subset was more challenging than other CMAPSS sub-datasets due to multiple operating conditions and two simultaneous fault modes (HPC degradation and fan degradation). This increased the difficulty of accurately modelling RUL, requiring explicit regime-aware preprocessing to prevent regime-induced sensor variation from masking true degradation signals.

Acoustic Anomaly Detection — Variability Across Machine Types

Different machine types in the MIMII dataset exhibited distinct acoustic characteristics and anomaly patterns. Some machines (e.g., valve) showed overlapping normal and abnormal reconstruction error distributions, making anomaly detection more difficult. PCA visualisation confirmed that MFCC features alone were insufficient for valve machines, motivating the addition of YAMNet embeddings and context windowing.

15.2 Future Work

- Cross-Subset Transfer Learning for RUL: Train on FD001, FD002, FD003 sub-datasets and fine-tune on FD004 to improve generalisation and robustness by leveraging knowledge from simpler operating conditions.
- Edge Deployment Optimisation: Convert trained LSTM and autoencoder models to TensorFlow Lite (TFLite) with quantisation to reduce model size and inference latency for real-time deployment on resource-constrained industrial devices.
- Multi-Machine Generalisation: Train a single autoencoder across all four MIMII machine types to evaluate cross-machine anomaly detection capability and enable a unified deployment model.
- Attention-Based RUL Prediction: Incorporate self-attention mechanisms or Transformer architectures to capture longer-range temporal dependencies in engine degradation patterns beyond what LSTM can model.
- Real-Time Streaming: Integrate the FastAPI backend with a streaming data source (e.g., Apache Kafka) to enable continuous online inference on live sensor and audio streams.

16. Conclusion

This document has specified the complete system analysis and design for an AI-powered predictive maintenance system comprising two integrated modules.

RUL Prediction Achievements

- Processed and analysed the CMAPSS FD004 dataset (61,249 training rows; 248 engine units; 249 test engines).
- Applied regime-aware preprocessing: K-Means clustering ($k=6$), per-regime MinMax scaling, CV-based and correlation-based sensor filtering.
- Developed a stacked LSTM model achieving test RMSE of 22.89 cycles and R^2 of 0.82.
- Applied TS2Vec self-supervised transfer learning as a feature extraction stage, achieving validation RMSE of 28.05 and test RMSE of 23.35.

Acoustic Anomaly Detection Achievements

- Processed machine sound data from the MIMII dataset for four machine types (valve, pump, fan, slide rail).
- Extracted MFCC (40-D) features and YAMNet (1024-D) transfer learning embeddings; concatenated into 1064-D vectors with 5-segment context windowing.
- Built a deep autoencoder (Encoder $4 \times 128 \rightarrow$ Bottleneck 8 $\rightarrow$ Decoder 4×128) trained exclusively on normal sound data.
- Applied reconstruction-error thresholding (statistical and percentile-based) with majority voting to achieve robust file-level anomaly classification.

Deployment & Experiment Management

- Integrated both modules into a unified FastAPI backend and Gradio interactive interface.
- Deployed the RUL prediction module on Hugging Face Spaces for real-time web-based inference.
- Used MLflow for comprehensive experiment tracking, model versioning, and performance comparison across all hyperparameter configurations.

The completed system demonstrates that combining regime-aware sensor preprocessing with LSTM deep learning, TS2Vec transfer learning, and autoencoder-based acoustic anomaly detection produces a robust, scalable, and deployable predictive maintenance solution suitable for industrial environments.